PROYECTO FORMATIVO · SENA

RaízVolt

Diseño y codificación de las API

Catálogo solar, inventario, cotizaciones y pedidos. Informe de diseño y guía de uso.

23 operaciones HTTP	2 roles de acceso	56 pruebas aprobadas

Aprendiz
Jhon Edwards Tineo Landaeta

Programa y ficha
Análisis y Desarrollo de Software · 3235874

Institución
Servicio Nacional de Aprendizaje (SENA)
CTMA - Regional Antioquia

Fecha
2 de noviembre de 2026 · Colombia

Instructor:
Diego Armando Barrera Araujo

Actividad AA5 ·EVO3 Servicios web del proyecto
Tecnologías: JavaScript, Node.js 24.x, SQLite, Git y OpenAPI.

01 / PROPÓSITO Y ALCANCE

Una API para el flujo comercial

Introducción

RaízVolt plantea comercializar soluciones solares mediante una empresa digital en Colombia. Esta evidencia reúne servicios para gestionar el catálogo y recorrer una venta: identificar al cliente, cotizar productos, reservar inventario y registrar el estado del pedido.

Objetivo general

Diseñar, codificar y documentar una API REST que permita consultar productos solares, controlar existencias y gestionar cotizaciones y pedidos con permisos diferenciados.

Objetivos específicos

- Definir recursos HTTP, entradas JSON, validaciones y respuestas.
- Persistir información y proteger contraseñas, sesiones y operaciones privadas.
- Evitar reservas duplicadas y mantener consistencia entre pedidos e inventario.
- Entregar código comentado, contratos, ejemplos, pruebas y repositorio.

Continuidad del proyecto

Antecedente revisado	Aplicación en esta entrega
AA5-EV01: registro e inicio de sesión	Se reutilizan auxiliares de contraseñas, errores y limitación de intentos. Se añaden tokens y roles.
AA4-EV03: catálogo React/Spring Boot	Se conservan campos de producto, cinco categorías y casos de registrar, consultar, actualizar y retirar.
Extensiones propuestas	Perfil, inventario con movimientos, cotizaciones y pedidos completan el flujo comercial. No se presentan como historias previamente aprobadas.

Delimitación: backend académico local. No procesa pagos, impuestos, flete, facturación electrónica, importaciones ni envíos reales. Tampoco migra H2 ni integra automáticamente el front-end React anterior.

02 / DISEÑO TÉCNICO

Arquitectura y datos

Se utiliza un monolito modular: una API HTTP con responsabilidades separadas. Node.js continúa la tecnología del ejercicio de autenticación; SQLite permite guardar información sin instalar un servidor de base de datos adicional.

Capa / archivos	Responsabilidad
HTTP · app.js	Rutas, permisos, lectura de JSON, códigos de respuesta y documentación.
Servicios · auth, catalog, sales	Reglas de cuentas, productos, inventario, cotizaciones y pedidos.
Validación · validation.js	Campos permitidos, tipos, longitudes, importes, IDs y paginación.
Persistencia · database.js y schema.sql	SQL parametrizado, índices, claves foráneas y transacciones.
Arranque · server.js y config.js	Puerto, ruta de datos, escucha local y cierre ordenado.

Modelo de datos: siete tablas

Tabla	Relaciones y propósito
usuarios	Identidad, rol y perfil. Un usuario tiene sesiones, cotizaciones y pedidos.
sesiones	Pertenece a un usuario. Guarda hash del token y vencimiento.
productos	Código único, categoría, precio en centavos, existencias y estado activo.
cotizaciones	Pertenece a un usuario; fecha, vigencia y total exacto.
cotizacion_items	Relaciona cotización y producto; guarda cantidad y copia de precio/nombre.
pedidos	Una cotización origina como máximo un pedido. Conserva dirección y estado.
movimientos	Producto, actor, variación, saldo, motivo y pedido relacionado opcional.

Los precios se guardan en centavos; las sumas usan BigInt y se devuelven como texto decimal. Las copias de precios y direcciones preservan la historia ante cambios del catálogo o perfil.

03 / REGLAS Y PERMISOS

Consistencia de la operación

Regla	Comportamiento implementado
Producto	Código 3-30; nombre 3-120; cinco categorías. Precio mayor que cero, hasta 9999999999.99 COP.
Inventario	Número entero de 0 a 1000000. Cada ajuste exige variación no nula y motivo de 5-200 caracteres.
Cotización	1-20 productos distintos; cantidad 1-10000 por producto; vigencia de siete días. No reserva existencias.
Pedido	Cotización propia vigente, perfil completo y productos activos con stock. Conserva el precio cotizado.
Transacción	La creación del pedido, las reservas y los movimientos se confirman juntos o se revierten juntos.
Reintentos	La misma cotización devuelve el pedido existente: no vuelve a descontar stock.
Cancelación	Solo desde PENDIENTE; devuelve unidades una vez. Una devolución que supera el máximo se rechaza completa.
Retiro	Baja lógica del producto. Conserva historia y código único; impide nuevas compras.

Estados del pedido

Cambio	Quién puede hacerlo
PENDIENTE → ENVIADO	ADMIN
ENVIADO → ENTREGADO	ADMIN
PENDIENTE → CANCELADO	Cliente propietario o ADMIN

Seguridad

El registro público crea CLIENTE. Solo un comando local crea ADMIN. Las contraseñas utilizan scrypt y sal individual; no se devuelven hashes ni claves. Los tokens son aleatorios, vencen en dos horas y se guardan como hash. Un nuevo ingreso revoca la sesión anterior.

Los clientes solo consultan sus cotizaciones y pedidos. La API valida en el servidor, usa consultas parametrizadas, limita el cuerpo a 32 KiB y admite 20 solicitudes de registro/login por IP cada diez minutos. Escucha únicamente en 127.0.0.1.

04 / GUÍA PRÁCTICA

Ejecutar y probar en Windows

Paso	Acción
1. Extraer	Clic derecho sobre el ZIP → Extraer todo. Abrir la carpeta API-PROYECTO.
2. Comprobar Node.js	En la terminal ejecutar node --version. Debe mostrar v24.x.x. Descarga oficial: https://nodejs.org/en/download.
3. Preparar demostración	Abrir PREPARAR_DEMO_WINDOWS.bat. Copiar el usuario y la contraseña ADMIN generados. Se agregan productos ficticios sin borrar datos existentes.
4. Iniciar	Abrir INICIAR_WINDOWS.bat y dejar la consola abierta. Visitar http://127.0.0.1:3000 para leer el manual.
5. Importar Postman	Importar ejemplos/RaizVolt.postman_collection.json. Completar admin_usuario y admin_contrasena en las variables de la colección.
6. Ejecutar solicitudes	Recorrer del paso 01 al 34 en orden o usar Collection Runner. Se crean un cliente, productos, cotizaciones y pedidos de prueba.
7. Detener	En la terminal del servidor presionar Ctrl+C. Los datos quedan en data/raizvolt.sqlite.

Desde Visual Studio Code

```
node --version node
scripts/preparar-demo.js node
src/server.js
```

No requiere npm install, MySQL ni Live Server. Para crear otro ADMIN: node scripts/crear-admin.js. Para ejecutar las pruebas: PROBAR_WINDOWS.bat. La página del navegador es documentación; Postman consume la API.

Si el puerto 3000 está ocupado

```
$env:PORT="3001"
node src/server.js
```

Estos dos comandos son para PowerShell. Después cambia base_url en Postman a http://127.0.0.1:3001. Consulta README.md para DB_PATH, respaldos y otras opciones.

05 / CATÁLOGO DE SERVICIOS

Acceso, perfiles y productos

Base: http://127.0.0.1:3000. Los servicios USUARIO requieren un token Bearer de CLIENTE o ADMIN. Las solicitudes con cuerpo usan Content-Type: application/json.

ID / acceso	Método y ruta	Resultado
S01 / PUBLICO	GET /api/salud	200 · Comprobar disponibilidad
S02 / PUBLICO	POST /api/registro	201 · Registrar cliente
S03 / PUBLICO	POST /api/login	200 · Iniciar sesión
S04 / USUARIO	POST /api/logout	204 · Cerrar sesión
S05 / USUARIO	GET /api/perfil	200 · Consultar perfil propio
S06 / USUARIO	PUT /api/perfil	200 · Completar o actualizar perfil
S07 / PUBLICO	GET /api/categorias	200 · Consultar categorías
S08 / PUBLICO	GET /api/productos	200 · Listar y buscar productos
S09 / PUBLICO	GET /api/productos/{id}	200 · Consultar producto
S10 / ADMIN	POST /api/productos	201 · Registrar producto
S11 / ADMIN	PUT /api/productos/{id}	200 · Actualizar producto
S12 / ADMIN	DELETE /api/productos/{id}	204 · Retirar producto

Ejemplo: registrar un cliente

```
POST /api/registro
Content-Type: application/json

{"usuario":"cliente.demo",
 "contrasena":"ClaveDemo2026!"}
```

Registro: 201. Login con el mismo cuerpo: POST /api/login, 200 y token. Credenciales incorrectas: 401. La contraseña mostrada es ficticia. No hay cuentas precargadas en el ZIP.

Para modificar existencias no se usa PUT del producto; se registra un movimiento mediante POST /api/productos/{id}/inventario.

06 / CATÁLOGO DE SERVICIOS

Inventario, cotizaciones y pedidos

ID / acceso	Método y ruta	Resultado
S13 / ADMIN	GET /api/inventario	200 · Consultar resumen de inventario
S14 / ADMIN	GET /api/inventario/movimientos	200 · Consultar movimientos
S15 / ADMIN	POST /api/productos/{id}/inventario	200 · Ajustar existencias
S16 / USUARIO	POST /api/cotizaciones	201 · Crear cotización
S17 / USUARIO	GET /api/cotizaciones	200 · Listar cotizaciones
S18 / USUARIO	GET /api/cotizaciones/{id}	200 · Consultar cotización
S19 / USUARIO	POST /api/pedidos	201 · Confirmar pedido
S20 / USUARIO	GET /api/pedidos	200 · Listar pedidos
S21 / USUARIO	GET /api/pedidos/{id}	200 · Consultar pedido
S22 / USUARIO	PATCH /api/pedidos/{id}/estado	200 · Cambiar estado del pedido
S23 / PUBLICO	GET /api/openapi.json	200 · Descargar contrato OpenAPI

Confirmar un pedido devuelve 201 la primera vez y 200 si esa cotización ya tiene pedido. Los listados usan pagina y limite. CLIENTE ve sus documentos; ADMIN puede consultar todos.

Ejemplo: cotizar y confirmar

```
POST /api/cotizaciones
Authorization: Bearer <token>
Content-Type: application/json

{"items":[{"productoId":1,"cantidad":2}]}

POST /api/pedidos
Authorization: Bearer <token>
Content-Type: application/json

{"cotizacionId":1}
```

Usar IDs reales de las respuestas. Completar antes el perfil mediante PUT /api/perfil. El servidor calcula el valor de los productos; no agrega flete ni impuestos.

07 / DOCUMENTACIÓN Y PRUEBAS

Verificación de entrega

Documentación de cada servicio

El manual completo está en docs/SERVICIOS.html (abrir con el navegador) y docs/SERVICIOS.md (editable). Cada operación incluye propósito, método, ruta, permiso, parámetros, cuerpo de ejemplo, respuesta y errores. docs/openapi.json ofrece el contrato estructurado. ENDPOINTS.txt resume todas las rutas.

Código HTTP	Uso principal
200 / 201 / 204	Operación correcta / recurso creado / respuesta sin cuerpo.
400 / 401 / 403	Entrada inválida / autenticación requerida / permiso insuficiente.
404 / 405 / 409	No encontrado / método incorrecto / conflicto de datos o de negocio.
413 / 415 / 429	Cuerpo grande / formato no admitido / límite de intentos.
500	Error interno sin trazas ni detalles SQL.

Resultado real de ejecución

56 pruebas aprobadas; 0 fallidas; 0 omitidas. Entorno de ejecución: Linux, Node.js 24.22.1, SQLite 3.53.4. Se utilizaron bases temporales y puertos libres. El registro TAP se conserva en docs/RESULTADOS_PRUEBAS.txt. `node --test tests/api.test.js tests/coleccion.test.js`

55 pruebas cubren autenticación, validaciones, catálogo, inventario, propiedad, vigencia, permisos, transacciones, concurrencia y persistencia. Una prueba adicional reproduce las 34 solicitudes de la colección y comprueba sus estados HTTP.

Precisiones: el recorrido de la colección se ejecutó por HTTP mediante Node.js; no se ejecutó la aplicación Postman ni se generaron capturas o un video. Los archivos .bat se suministran para Windows; la ejecución automatizada registrada se realizó en Linux.

Las pruebas demuestran el comportamiento local descrito. No certifican carga masiva, seguridad productiva ni integración con el front-end React anterior.

Fuentes, repositorio y archivos

Repositorio

https://github.com/Joan-dot650/Programa-Git

Rama publicada: evidencia-aa5-api-proyecto
Carpeta: API-PROYECTO
Verificación: VERSIONAMIENTO.txt del ZIP identifica las revisiones remota y local.

```
git clone --branch evidencia-aa5-api-proyecto
versionamiento/RaizVolt.bundle Proyecto_Git
cd Proyecto_Git/API-PROYECTO git log --
oneline -- . git status
```

El código y la documentación están publicados en la rama indicada, carpeta API-PROYECTO.
ENLACE_REPOSITORIO.txt contiene el enlace directo. El comando anterior recupera el historial local sin internet: escribir git clone en una sola línea desde la carpeta extraída del ZIP.

Contenido entregado

Elemento	Ubicación
Archivos del proyecto	API-PROYECTO/src, scripts, tests y archivos de ejecución.
Documentación de servicios	API-PROYECTO/docs: PDF, HTML, Markdown y OpenAPI.
Ejemplos de consumo	API-PROYECTO/ejemplos: colección Postman y peticiones HTTP.
Enlace e historial	ENLACE_REPOSITORIO.txt, VERSIONAMIENTO.txt y versionamiento/RaizVolt.bundle.
Nombre del comprimido	JHON_EDWARDS_TINEO_LANDAETA_AA5_EV03.zip.

Numeración: se respetó AA5_EV01 tal como aparece en el texto de la guia., se renombro el ZIP y su carpeta contenedora como AA5-EVO3 como aparece en el documento de evaluacion; Esta entrega se conserva separada de la evidencia anterior.

Referencias

Node.js, SQLite (documentación oficial de la rama 24): https://nodejs.org/download/release/latest-v24.x/docs/api/sqlite.html

OpenAPI Initiative, especificación 3.1.1: https://spec.openapis.org/oas/v3.1.1.html

OWASP, Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Consulta: 2 de noviembre de 2026. Antecedentes del proyecto: AA5-EV01 y documentación del catálogo AA4-EV03.